

NELSON MANDELA METROPOLITAN UNIVERSITY
SUMMERSTRAND CAMPUS (SOUTH)

SEMESTER TEST QUESTION PAPER

29 APRIL 2016

SUBJECT: Advanced Programming 3.1

MODULE CODE: WRAP301

TIME : 2½ hours

MARKS: 70

EXAMINERS:

MODERATORS:

1. Dr. D. Vogts

1. Dr. K.A. Naudé

SPECIAL INSTRUCTIONS

1. Answer **ALL** questions.
2. Permitted materials:
 - JDK online documentation (<http://docs.oracle.com/javase/8/docs/api/> or on C:).
 - Bound lecture notes.
 - Online lecture notes (//cs2/courses/wrap301/WRAP301 Notes.pdf).
 - Clean Java text book (does not have to be prescribed one, but may not be written in).
 - Internet access is allowed, **but** file repositories and online chatting are expressly forbidden. Forums, developer sites, API documentation, Google, etc. are allowed (read only). Misuse of the internet during the test is a punishable offence – if unsure, ask!
3. Your solutions for the questions in this paper **must** be your own original work.
4. **Each of your source files (*.java) must include your name and student number at the top of each listing (as a comment).**
5. You must place a copy of your code on the T: drive in the sub-folder allocated to you.

Note: You may use your lecture notes, text book and JavaDocs and Internet (read conditions on front page) when answering these questions!

- 1) Resource management with many applications requires that the full file path to a resource be stored, e.g. `/assets/images/presentation1/screenshot01.png` or `/assets/images/presentation2/photo.png`. In many cases, the path name can be quite lengthy and only given the file's name and extension, e.g. `screenshot01.png`, it is difficult to determine in which folder the file resides.

Dungeon Siege¹ came up with a solution in which folder path information was encoded into the name of the file itself, so that only the file's name and extension needed to be saved and not any of the folders. Using only the file's name, it was possible to determine which folder the file resided in. This encoding was known as an NNK Filename². The encoding works by prepending the file's name with abbreviations of folder names separated by underscores. The original filenames were then renamed to the encoded ones.

For example, the NNK Filename for the two files might be `a_i_p01_screenshot01.png` and `a_i_p02_photo.png` respectively. The folder abbreviations used would be:

Folder Abbreviation	Folder
a	assets
a_i	images
a_i_p01	presentation1
a_i_p02	presentation2

Using NNK Filenames, `a_i_p01_screenshot01.png` and `a_i_p02_photo.png` would be the new filenames and these would be stored instead of having to store `/assets/images/presentation1/a_i_p01_screenshot01.png` and `/assets/images/presentation2/a_i_p02_photo.png` respectively.

The full file path can be obtained by examining the prefixes of a file's name (i.e. each part separated by an underscore) and building up the full file path. For example, to get the full file path for `a_i_p01_screenshot01.png`, the following procedure could be followed:

1. `path = ""`
2. 1st folder abbreviation is `a`, so `path = path + "/assets"`
3. 1st and 2nd folder abbreviation is `a_i`, so `path = path + "/images" = "/assets/images"`
4. 1st, 2nd and 3rd folder abbreviation is `a_i_p01`, so `path = path + "/presentation1" = "/assets/images/presentation1"`
5. No more abbreviations, so full path of file is :
`"/assets/images/presentation1/a_i_p01_screenshot01.png"`

- 1.1) Write an implementation of a class called `NNKFilename` that: (13)

- has a static method called `addFolderAbbreviation` which accepts a folder abbreviation and the folder name and adds this to the list of accepted folder abbreviations (the table);
- has a static method called `decode` which accepts an NNK Filename, decodes it using the folder abbreviations and returns the full file path. If invalid folder abbreviations are used, then an empty string should be returned.

E.g. `decode("a_i_p01_screenshot01.png")` should return
`"/assets/images/presentation1/a_i_p01_screenshot01.png"`

Be sure to write appropriate pre- and post-conditions for each method, as well as make use of assertions to ensure that these are met. Make sure that your code is properly commented.

¹ <http://store.steampowered.com/app/39190/>

² <http://www.siegenetwork.com/forums/viewtopic.php?t=7779>

- 1.2) Write a JUnit suite of tests, using the Glass Box strategy³, to test the methods you wrote in (1.1). Ensure that your tests adequately test for boundary conditions and code coverage. (12)

- 2) Write a ***console application*** that: (12)

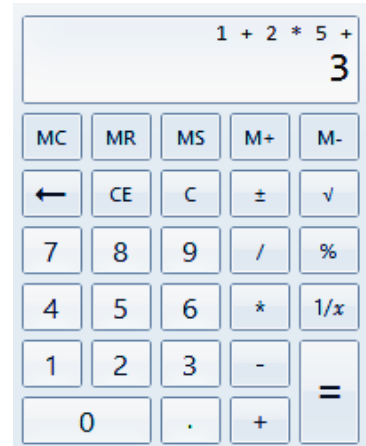
1. Connects to the addresses database found on the SQL Server. The server name is //OPENBOX\WRR and the database name is "WRAP301ST2". The username is "ST2User" and the password is "May2015".
2. Queries the database for the surnames of customers where there is more than one customer with the same surname.
3. Displays the surnames obtained in #2.
4. Terminates the connection with the database

- 3) A screenshot of the *standard* calculator provided by Windows is shown to the right. Play around with the calculator to see how it functions.

Note specifically that it *does not* use BODMAS, i.e. operations are performed in the order that they occur – multiplication does not happen before addition as a rule, unless the multiplication happened to appear first. In other words using the standard calculator, $1 + 2 * 5 = 15$, not 11.

For this question, you will be required to implement a subset of the functionality provided by the standard calculator using JavaFX.

Hint: pay special attention to what is happening with every button press, i.e. what is displayed in the number display, what happens if you press the same button multiple times, etc.



- 3.1) Create a UI that looks similar to that of the standard calculator. It is not necessary to style the controls (like the section showing the sequence of operations entered or the answer). The special symbols used, i.e. $\leftarrow$, $\pm$ and $\sqrt{}$, can be replaced with simple text strings, namely "<=", "+/-" and "sqrt" respectively. (9)

- 3.2) Implement the functionality of the calculator, but *only* for the following buttons: (24)

- numbers,
- point/dot,
- plus,
- multiply,
- equals and
- clear.

Once completed:

- **Remember to put your name and student number at the top of each *.java file.**
- **NO PRINTING!**
- **SAVE your work to the T:**
- **Remember that if you do not save all your work to the T:, there is nothing to mark, i.e. expect 0!**
- **You may not leave the venue until the last 15 minutes of the test.**

³ Also known as the White Box strategy.